

Charotar University of Science and Technology (CHARUSAT)
Faculty of Technology and Engineering (FTE)

Subject: Object Oriented Programming (CEUE203)

Semester: 3rd

Teaching Scheme:

Teaching Scheme	Theory	Practical	Tutorial	Total	Credit
Hours/Week	2	2	-	4	3
Marks	50	50	-	100	

Course Pre-requisites:

- Basic understanding of programming concepts and familiarity with Object-Oriented Programming (OOP) principles.

Course Description:

This course provides a comprehensive introduction to the Java programming language, equipping students with a solid foundation in both its theoretical concepts and practical applications. It begins with fundamental topics such as Java syntax, data types, and control structures. The curriculum then delves into object-oriented programming principles, including classes, objects, inheritance, and polymorphism. Students will explore key Java features like exception handling, the Collections Framework, and input/output operations. The course also introduces advanced concepts such as multithreading and concurrency, ensuring participants gain the skills necessary for developing robust Java applications.

Course Objectives:

1. To understand the fundamentals of Java programming, including syntax, data types, and control structures, to build a strong programming foundation.
2. To learn the principles of Object-Oriented Programming (OOP) through classes, inheritance, interfaces, and packages, enabling modular and reusable code design.
3. To develop proficiency in handling arrays and strings for efficient data manipulation and processing in Java applications.
4. To explore advanced concepts such as multithreaded programming, exception handling, and file I/O operations, ensuring robust and concurrent application development.
5. To master the Java Collections Framework and generics for effective data management and implementation of reusable, type-safe code structures.

Course Outcomes:

By the end of the course, students will be able to:

1. **CO1:** Demonstrate a comprehensive understanding of Java fundamentals, including syntax, data types, control structures, and object-oriented programming principles

such as inheritance, polymorphism, abstraction, and encapsulation.

2. **CO2:** Utilize inheritance, interfaces, and Java package structures to create modular and maintainable code, promoting code reusability and efficient project organization while adhering to object-oriented design principles.
3. **CO3:** Construct and handle exceptions proficiently to build fault-tolerant applications, applying exception handling mechanisms to manage errors and maintain program stability.
4. **CO4:** Develop and implement multi-threaded applications using Java's built-in thread handling mechanisms and concurrency utilities to optimize program performance and resource utilization.
5. **CO5:** Design and implement Java programs using file handling, the NIO.2 package, and Stream API to perform efficient file operations and process data streams effectively, enhancing data-driven application capabilities.
6. **CO6:** Apply the concepts of the Collection Framework and Generics to create robust and efficient programs that effectively manage and manipulate data, ensuring type safety and reducing runtime errors.

➤ **Self-study/Further Study components and materials:**

Unit No.	Unit/Topic Details	
1	INTRODUCTION	
	1.1	Overview of OOP principles: Encapsulation, Inheritance, Polymorphism, Abstraction.
	1.2	Data Types: Integer (byte, short, int, long), Floating-point (float, double), Character, Boolean.
	1.3	Variables: Dynamic Initialization, Scope and Lifetime of variable, Type Casting, Type promotion rule
	1.4	Operators: Arithmetic, Bitwise, Relational, Boolean, Assignment, ternary (three-way).
	1.5	Control Statements: if, nested ifs, Traditional Switch, Iteration statements (while, do-while, for, For-Each, Nested Loops), Jump statements (break, continue, return)
	Materials:- ➤ https://www.geeksforgeeks.org/operators-in-java/	
2	BASICS OF ARRAY	
	2.1	Introduction to Arrays, Types of Array
	2.2	Array Operations: Traversing arrays with for, for-each, and enhanced for loop
	2.3	Copying arrays, cloning arrays, Searching and sorting elements in an array, Jagged Arrays, Memory Representation of Arrays
	Materials:- ➤ https://www.programiz.com/java-programming/arrays ➤ https://www.geeksforgeeks.org/jagged-array-in-java/ ➤ https://www.geeksforgeeks.org/where-is-the-memory-allocated-for-arrays-in-java/	
3	FUNDAMENTAL OF INHERITANCE	
	3.1	Concept and need for inheritance in OOP, Types of Inheritance: Single Inheritance, Multilevel Inheritance, Hierarchical Inheritance
	Materials:- ➤ https://www.scaler.com/topics/java/inheritance-in-java/	

	FUNDAMENTAL OF POLYMORPHISM	
	3.2	Concept and need for polymorphism in OOP, Types of polymorphism, Method Overloading, Method Overriding
	Materials:- ➤ https://www.scaler.com/topics/java/polymorphism-in-java/	

Syllabus:

Unit No.	Unit/Topic Details		Hours (hr)	Evaluation Weightage (%)
1	FUNDAMENTAL OF PROGRAMMING IN JAVA		02	06
	1.1	History of Java.		
	1.2	Java's Magic: The Bytecode.		
	1.3	The Java Buzzwords: Simple, Object-Oriented, Robust Multithreaded, Architecture-Neutral, Interpreted and High Performance, Distributed, Dynamic.		
	1.4	Switch Expressions, Records, and Other Recently Added Features.		
2	CLASS FUNDAMENTALS		04	14
	2.1	Class Fundamentals: Defining classes and creating objects.		
	2.2	Instance and static variables, methods, and the new keyword, Assigning Object Reference Variables.		
	2.3	Access modifiers: public, private, protected, default (package-private).		
	2.4	Using final with classes, methods, and variables, Understanding the this keyword.		
	2.5	Constructors: Types of constructors: default and parameterized, Constructor overloading, instanceof operator, The role of the this() constructor call.		
	2.6	Nested and Inner Classes: static nested class, inner class, method-local inner class, and anonymous inner class		
	2.7	Understanding the Object Class: The Object class and its methods: equals(), hashCode(), toString(), clone(), etc. Overriding toString() and equals() in custom classes.		
	2.8	Memory Management in Java: Stack vs. heap memory, Object lifecycle and garbage collection, Manual memory management and implications.		
	2.9	Java Annotations and Meta-annotation: Overview and purpose of annotations, Custom annotation, Reflection in Java.		
3	STRING HANDLING		02	06
	3.1	Introduction to Strings, Methods of the String class, StringBuffer and StringBuilder, String Manipulation Techniques, Regular Expressions (Regex), String Tokenization.		
4	INHERITANCE, INTERFACES & PACKAGES		05	17

	4.1	Understanding super keyword, Dynamic method dispatch and runtime polymorphism, final Keyword in Inheritance.		
	4.2	Abstract Classes and Methods, Casting Objects in Inheritance.		
	4.3	Introduction to Interfaces, Interface Segregation Principle, Interface Methods and Variables, Multiple Inheritance Using Interfaces, default and static method, private interface, Functional Interfaces, Marker Interfaces.		
	4.4	Introduction to Packages, Access Modifiers and Packages, Creating and Importing Packages, Static Imports, Jar Files. Introduction to the Java Module System, Creating and Declaring a Module, Understanding Module Descriptors, Working with Multiple Modules, Services and Service Loaders.		
5	EXCEPTIONS HANDLING		02	06
	5.1	Introduction to Exception Handling, Types of Exceptions: Checked Exceptions, Unchecked Exceptions, Try-Catch-Finally block.		
	5.2	Throw and Throws keyword, Try-With-Resources Statement.		
	5.3	Try-With-Resources Statement, Custom Exceptions, Exception Propagation.		
6	MULTITHREADED PROGRAMMING		05	17
	6.1	Introduction to Multithreading, Life cycle of a thread.		
	6.2	Understanding of Thread class and the Runnable interface, Organizing threads into groups.		
	6.3	Thread Synchronization, Inter-thread Communication, Thread Priority and Scheduling, Deadlock and Thread Safety Issues.		
	6.4	Executor and ExecutorService for thread pool management, Fork/Join Framework.		
7	FILE NIO		04	14
	7.1	Path Interface, Files Class, File Attributes.		
	7.2	Byte and Character Stream, Serialization.		
	7.3	NIO Fundamental: Buffer, Channel, Selector. Directory Stream and File Walking.		
8	COLLECTION FRAMEWORK AND GENERICS		06	20
	8.1	Introduction to Collection Framework, Collection Interface.		
	8.2	List Interface: ArrayList, LinkedList and Vector and Stack.		
	8.3	Set Interface: HashSet, LinkedHashSet and TreeSet.		
	8.4	Map Interface: HashMap, LinkedHashMap, TreeMap and Hashtable.		
	8.5	Working with Collections: Iterating through Collections, Sorting and Searching, Thread-safe Collections.		

	8.6	Introduction to Generics, Generic Classes and Methods, Bounded Type Parameters, Generics and Wildcards, Type Erasure.		
Total Hours:			30	

Course Articulation Matrix:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	2	3	3	-	-	-	-	-	-	-	-	-	-	-
CO2	3	2	3	2	1	1	-	-	-	-	-	-	1	3
CO3	-	2	2	3	2	-	-	-	-	-	1	1	-	-
CO4	1	1	1	1	3	-	-	-	-	-	2	2	3	-
CO5	1	1	2	3	3	-	-	1	-	-	1	1	2	2
CO6	1	1	1	2	3	1	-	1	-	-	2	1	3	3

Enter correlation levels 1, 2 or 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High), If there is no correlation, put “-”

Recommended Study Material:

❖ Text books:

1. “Java: The Complete Reference, Thirteenth Edition”, by Herbert Schildt, McGraw Hill.
2. “OCP Oracle Certified Professional Java SE 17 Developer (Exam 1Z0-829) Programmer's Guide”, by Khalid A. Mughal, Vasily A. Strelnikov, Oracle Press.

❖ Reference Books:

1. “Effective Java, 3rd Edition”, by Joshua Bloch, Addison-Wesley Professional.
2. “Core Java Volume I--Fundamentals (Core Series)”, by Cay S. Horstmann, Pearson.
3. “Head First Java, 3rd Edition”, by Kathy Sierra, Bert Bates, Trisha Gee, O'Reilly Media.

Online resources:

1. <https://docs.oracle.com>
2. <https://www.baeldung.com/>
3. <https://dev.java/>

Care-taking Points:

- Doubts arising from self-study topics will be addressed and clarified during classroom sessions to ensure comprehensive understanding.
- Ensure that theoretical concepts are backed by hands-on practice. Every major concept, like inheritance, interface, multithreading, stream API, collections framework etc., should be complemented by coding exercises or projects to solidify understanding.
- Teach coding best practices, including clean code principles, design patterns, and proper use of Java's built-in utilities. This ensures students learn to write efficient,

maintainable, and scalable code.

Practical Skill Enhancement Points:

- Practical enhancement points bridge the gap between theoretical concepts and real-world application, enabling students to gain hands-on experience in solving complex problems
 - Develop the ability to implement object-oriented programming principles to design and create modular, reusable, and scalable Java applications.
 - Gain hands-on experience with Java Collections, Generics, and multithreading for managing data structures and achieving concurrency in real-world scenarios.
 - Enhance problem-solving skills by building programs that handle exceptions, manage file I/O, and process input/output operations efficiently.
 - Acquire practical expertise in creating and debugging Java applications that integrate arrays, strings, and advanced features like NIO and interfaces.